EMOTICORE SYSTEMS · PROJECT DOCUMENTATION

EMOTICORE — Project Report

AI-powered behavioral threat detection, running entirely in the browser. Complete record of what was built, the technologies and models used, the algorithms behind each feature, and the decisions that shaped them.

VERSION 1.0 JULY 2026 9 GIT COMMITS ~5,400 LINES 4 PAGES + PWA

---

01 Executive overview

Emoticore is a product concept for an AI security platform that fuses live camera feeds and speech analysis into a single behavioral threat picture — "Detect intent before it becomes incident." This project implements it as a complete, working website: an enterprise-grade marketing page plus **three real, functioning product surfaces** where the AI genuinely runs.

Everything is built under four deliberate constraints:

- **No backend.** Every page is a self-contained static HTML file — no server, no database, no accounts.
- **No frameworks, no build step.** Pure HTML5 / CSS3 / vanilla JavaScript. The files open directly in a browser.
- **On-device analysis.** Video and audio never leave the machine. The single exception (optional speech transcription) is opt-in and labeled in the UI.
- **One design system.** A "defense-tech mission control" visual identity shared across all pages.

02 What was built — the four pages

FILE	ROLE	SIZE
`index.html`	Enterprise landing page (marketing + simulated console)	1,405 lines
`demo.html`	Live camera demo — real facial emotion detection	944 lines
`voice.html`	Voice emotion analysis — record/upload a clip, get an emotion estimate	1,158 lines
`console.html`	Operations Console — the full product: fusion, sessions, incidents, reports	1,825 lines
`sw.js` · `manifest.webmanifest` · `icon.svg`	PWA layer — installable app, offline shell	75 lines

index.html — Landing page

- Animated hero: particle-constellation canvas, glitch-effect brandline, staggered load choreography, animated stat counters (99.2%, <400ms, 140+).
- Simulated operations console: canvas camera-feed simulation with wandering subject lock, drifting telemetry bars, typing event log with UTC timestamps.
- Capability rack (Speech / Face / Scoring bays with live canvas visuals), scroll-drawn pipeline section, fictional-client trust marquee, big ops-dashboard mockup with SVG threat gauge, deployments, pull-quote, CTA band, 5-column footer.
- Scroll-reveal system via IntersectionObserver; fully responsive with a mobile hamburger menu; respects `prefers-reduced-motion`.

demo.html — Live camera demo

- Real in-browser ML: face detection, 68 facial landmarks, and 7-class expression classification on the user's webcam.
- Live overlay: corner-bracket subject lock, landmark dots, per-subject labels; **stable subject IDs** via frame-to-frame IoU tracking.
- Smoothed affect telemetry bars, weighted facial threat index on an SVG gauge, event log, FPS/latency stats, and a 90-second threat sparkline.
- Full permission flow with distinct error states (denied / no camera / engine load failure) and a localhost hint for `file://` quirks.

voice.html — Voice emotion analysis

- Record up to 15 s of microphone audio (codec-free PCM capture) or upload an audio file (MP3/WAV/M4A/OGG, first 30 s).
- On-device prosody engine scores six emotions — Calm, Happy, Surprised, Nervous/Fearful, Angry, Sad — plus a 0–100 vocal stress index.

- Outputs: dominant-emotion verdict in plain language, measured prosodic features (mean pitch, range, jitter, energy, rate, pauses), waveform envelope, WAV replay of the sample.
- Last five analyses persist in `localStorage`; click any history entry to re-view. Rejects samples with under ~2 s of audible speech.

console.html — Operations Console (flagship)

- **Fusion pipeline:** camera and voice analyzed simultaneously; facial threat and vocal stress merge into one live index, with transcript keyword hits adding a decaying boost. Mic denied → graceful face-only mode.
- **Session recording:** telemetry sampled at 4 Hz into a live 120-second scrolling timeline chart with threshold bands, three plotted series, a dominant-emotion color strip, and a time grid.
- **Incident engine:** threat held above ELEVATED for 1.5 s opens an incident — video-frame snapshot captured, live card with peak/duration/driver, severity escalation to HIGH, hysteresis close.
- **Session reports:** end-of-session summary (avg/peak threat, % time elevated, affect distribution) with JSON and CSV export — generated locally as Blob downloads.
- **Settings drawer:** face/voice fusion balance, both thresholds, editable threat lexicon, alert-sound toggle — persisted in `localStorage`.
- **Command palette:** Ctrl+K, filterable, keyboard-navigable — sessions, toggles, exports, navigation.

03 Technology stack

LAYER	TECHNOLOGY	NOTES
Languages	HTML5, CSS3, vanilla JavaScript	Zero frameworks, zero build tools, zero npm dependencies
Face AI	`@vladmandic/face-api` (TensorFlow.js-based)	Loaded from jsDelivr CDN on demand (~4 MB engine + model weights, first run only)
Audio DSP	Web Audio API	AnalyserNode, ScriptProcessorNode, custom autocorrelation pitch detection — all hand-written
Transcription	Web Speech API	Chrome/Edge only; routes audio via the browser's cloud speech service — strictly opt-in and labeled
Graphics	Canvas 2D + inline SVG	All visuals generated in code: particle mesh, feed simulation, waveforms, charts, gauges
Typography	Google Fonts	Sora (display), IBM Plex Sans (body), IBM Plex Mono (telemetry)
Storage	`localStorage` + Blob downloads	Settings, voice history; JSON/CSV report exports
PWA	Web App Manifest + Service Worker	Stale-while-revalidate caching; installable; registers only over https/localhost
Version control	Git (local repository)	9 commits, one per milestone; GitHub connection planned but not yet made
Development	Claude Code (VS Code extension), Fable model	AI-assisted build; code-level self-review each milestone

04 AI models & analysis engines

4.1 Facial pipeline — neural networks (demo.html, console.html)

MODEL	TASK	OUTPUT
`tinyFaceDetector`	Face detection (Tiny-YOLO-style)	Bounding boxes + confidence; input 416 px, threshold 0.4
`faceLandmark68TinyNet`	Facial landmarks	68 landmark points per face
`faceExpressionNet`	Expression classification	7 classes: neutral, happy, sad, angry, fearful, disgusted, surprised (probabilities)

Models are fetched once from CDN, then run entirely locally at typically 10–25 inference FPS. Emotion values are EMA-smoothed ($\alpha = 0.25$) before display.

4.2 Vocal pipeline — signal processing (voice.html, console.html)

- **Pitch (F0):** autocorrelation over a ~8 kHz decimated copy of the signal, 60–400 Hz search range, confidence-gated. Decimation cuts the ACF cost ~40× so a 15 s clip analyzes in well under a second.
- **Loudness:** per-frame RMS → dB; **jitter:** frame-to-frame F0 delta / mean; **rate:** energy-peak (syllable-proxy) count per second; **pauses:** sub-threshold frame ratio; **contour:** linear-fit F0 slope.
- **Live vocal stress (console):** a per-session baseline is calibrated from the speaker's first ~30 voiced frames (median F0); stress then measures deviation — pitch elevation + jitter + loudness spikes vs. that personal baseline.
- **Clip emotion estimate (voice page):** extracted features are normalized and combined through per-emotion weighted scoring functions, floored and normalized to percentages across the six categories.

4.3 Threat fusion (console.html)

```
facial_raw = 100 × min(1, angry×1.00 + fearful×0.85 + disgusted×0.50 + sad×0.20 +
surprised×0.25)
fused = (faceW × facial_raw + voxW × vocal_stress) / (weights present) + lexicon_boost
threat_index ← threat_index + (fused − threat_index) × 0.12  // EMA smoothing

defaults: faceW = 0.65, voxW = 0.35 · lexicon_boost ≤ 45, decays ×0.988/frame
levels: LOW < 35 ≤ ELEVATED < 65 ≤ HIGH (all user-tunable)
```

4.4 Incident engine (console.html)

```
open:  threat ≥ ELEVATED sustained ≥ 1.5 s → capture snapshot, open card
close: threat < ELEVATED − 5 sustained ≥ 2.0 s  // hysteresis prevents flicker
escalate: peak ≥ HIGH → severity HIGH
```

4.5 Subject tracking (demo.html)

Greedy intersection-over-union matching of face boxes between consecutive frames (IoU > 0.2, best-pair-first); unmatched detections get new IDs; tracks expire after 1.2 s unseen. Result: `SUBJ_01` / `SUBJ_02` labels stay attached to the same person.

05 Design system

Direction: **"defense-tech mission control"** — instrument panels, hairline borders, monospace telemetry; restrained glow rather than neon-everything. Single dark theme, committed deliberately.

Color tokens

`bg #05070d` `panel #0b101c → #0d1424` `ink #e8edf6` `cyan #38e1ff` `cyan-hot #00f5ff`
`violet #8b5cf6` `ok #34d399` `warn #fbbf24` `crit #f87171`

Typography

- **Sora** (600–800) — display headlines; geometric, machine-vision feel.
- **IBM Plex Sans** (400–600) — body; enterprise-infrastructure DNA.
- **IBM Plex Mono** — all telemetry, timestamps, labels; `tabular-nums` everywhere digits align.

Signature components (shared across pages)

The 270° SVG threat gauge (r = 42, dasharray 198/264, rotated 135°) · telemetry bar rows · timestamped event log · state pills · toggle chips · console/dash panel shells · corner-bracket subject lock. Animation is orchestrated (hero load sequence, scroll reveals) and `prefers-reduced-motion` is respected globally.

06 Privacy architecture

- All video inference and all audio DSP run **on-device**; no frame or sample is ever transmitted. Every page states this in the UI.
- The one exception is **transcription**: the Web Speech API routes audio through the browser vendor's speech service. It is off by default, opt-in per session, and labeled as cloud-routed at the moment of enabling.
- Cameras and microphones are released on stop and on page exit (`pagehide`).
- Exports (JSON/CSV) are generated locally as Blob downloads — no upload endpoint exists.

07 Development record

COMMIT	MILESTONE
`7838ae7`	Initial landing page (single-file futuristic design, simulated console)
`cb236f1`	Live webcam emotion demo — real ML in the browser
`e75f95c`	Microphone/speech module fused into the camera demo
`4b3d331`	Voice analysis separated into standalone page; Request-demo CTAs replaced with live demos
`72e2678`	Operations Console — sessions, timeline, incidents, exports, settings, palette
`cbf439e`	Site-wide polish — favicon, meta/OG tags, console integration
`6aaa726`	Repository hygiene (untrack local tool settings)
`7fd69d8`	Threat sparkline, stable subject IDs, voice analysis history
`87dd58d`	PWA — manifest, icon, service worker

Each feature milestone included a code-level self-review pass before commit. Bugs caught this way before they ever reached the browser include: an autocorrelation cost explosion on long clips (~10^9 operations, fixed by 8 kHz decimation), invisible telemetry bars (CSS dimensions on inline elements), a visible seam in the marquee loop, ring animations expanding off-center, and a stuck keyword boost when the mic was muted.

08 Honest limitations

> **Demo-grade accuracy.** The face models are open, lightweight networks — excellent for a pitch, not the "99.2%" of the marketing copy. The vocal stress and voice-emotion scores are transparent DSP heuristics (prosody distinguishes arousal well, valence roughly), labeled "not a clinical measure" in the UI. A production system would need serious models, validation datasets, and a legal/compliance review — emotion recognition is regulated in some jurisdictions (e.g. the EU AI Act).

- Session data lives in memory; closing the tab discards an unexported session.
- Transcription requires Chrome/Edge and an internet connection; Firefox shows "N/A".
- Camera/mic on `file://` work in most Chromium browsers, but localhost/https (VS Code Live Server) is the reliable path — and required for the PWA/service worker.
- First run needs internet for the ~4 MB engine + models (cached afterward).

09 Roadmap candidates

1. **Deploy to GitHub Pages** — real shareable https URL; camera, mic, and PWA install all work properly. Blocked only on GitHub sign-in.

2. **Session persistence** — auto-save sessions to IndexedDB; session browser/compare view.
3. **Neural voice emotion model** — upgrade the prosody heuristic when a browser-ready speech-emotion model (e.g. a wav2vec2 ONNX conversion for transformers.js) is available.
4. **Multi-camera console** — several streams per console with a combined incident rail.
5. **Docs page** — public-facing architecture/API documentation in the same design system.
